

TUGAS BESAR

STRATEGI ALGORITMA

**Pemanfaatan Algoritma Greedy dalam pembuatan bot
permainan Robocode Tank Royale**



Dosen pengampu: Imam Eko Wicaksono [S.Si.](#), [M.Si](#)

Asisten Dosen: Anisah Octa Ria

Kelas: RB

Disusun Oleh:

“OMKEGAS”

Dimas Alhamdy(124140165)

Rajendra Rifandhy Anandariantanto(124140099)

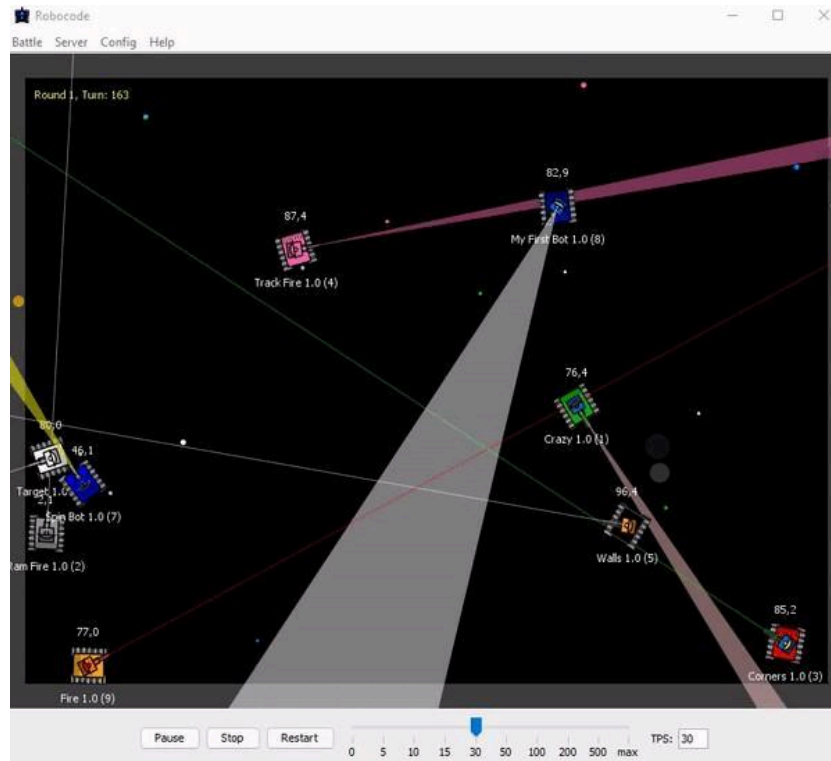
Najib Abhinaya(124140124)

PROGRAM STUDI TEKNIK INFORMATIKA
INSTITUT TEKNOLOGI SUMATERA

2026

BAB 1

DESKRIPSI TUGAS



Gambar 1. Robocode Tank Royale

Robocode adalah permainan pemrograman yang bertujuan untuk membuat kode bot dalam bentuk tank virtual untuk berkompetisi melawan bot lain di arena. Pertempuran Robocode berlangsung hingga bot-bot bertarung hanya tersisa satu seperti permainan Battle Royale, karena itulah permainan ini dinamakan Tank Royale. Nama Robocode adalah singkatan dari "Robot code," yang berasal dari [versi asli/pertama permainan ini](#). Robocode Tank Royale adalah evolusi/versi berikutnya dari permainan ini, di mana bot dapat berpartisipasi melalui Internet/jaringan. Dalam permainan ini, pemain berperan sebagai programmer bot dan tidak memiliki kendali langsung atas permainan. Pemain hanya bertugas untuk membuat program yang menentukan logika atau "otak" bot. Program yang dibuat akan berisi instruksi tentang cara bot bergerak, mendeteksi bot lawan, menembakkan senjatanya, serta bagaimana bot bereaksi terhadap berbagai kejadian selama pertempuran.

Pada Tugas Besar pertama Strategi Algoritma ini, mahasiswa diminta untuk membuat sebuah bot yang nantinya akan dipertandingkan satu sama lain. Tentunya mahasiswa harus menggunakan **strategi greedy** dalam membuat bot ini.

Komponen-komponen dari permainan ini antara lain:

1. Rounds dan Turns

Pertempuran dapat terdiri dari beberapa rounds. Secara default, satu pertempuran berisi 10 rounds, di mana setiap rounds akan memiliki pemenang dan yang kalah.

Setiap round dibagi menjadi beberapa turns, yang merupakan unit waktu terkecil. Satu turn adalah satu ketukan waktu dan satu putaran permainan. Jumlah turn dalam satu round tergantung pada berapa lama waktu yang dibutuhkan hingga hanya tersisa bot terakhir yang bertahan.

Pada setiap turn, sebuah bot dapat:

- Menggerakkan bot, memindai musuh, dan menembakkan senjata.
- Bereaksi terhadap peristiwa seperti saat bot terkena peluru atau bertabrakan dengan bot lain atau dinding.
- Perintah untuk bergerak, berputar, memindai, menembak, dan sebagainya dikirim ke server untuk setiap turn.

Perlu diperhatikan bahwa [API \(Application Programming Interface\)](#) bot resmi secara otomatis mengirimkan niat bot ke server di balik layar, sehingga Anda tidak perlu mengkhawatirkannya, kecuali jika Anda membuat API Bot sendiri.

Pada setiap turn, bot akan secara otomatis menerima informasi terbaru tentang posisinya dan orientasinya di medan perang. Bot juga akan mendapatkan informasi tentang bot musuh ketika mereka terdeteksi oleh pemindai.

Perlu diketahui bahwa game engine yang akan digunakan pada tugas besar ini tidak mengikuti aturan default mengenai komponen Round & Turns.

2. Batas Waktu Giliran

Penting untuk dicatat bahwa setiap bot memiliki batas waktu untuk setiap turn yang disebut turn timeout, biasanya antara 30-50 ms (dapat diatur sebagai aturan pertempuran). Ini berarti bahwa bot tidak bisa mengambil waktu sebanyak yang mereka inginkan untuk bergerak dan menyelesaikan turn saat ini.

Setiap kali turn baru dimulai, penghitung waktu ulang diatur ulang dan mulai berjalan. Jika batas waktu tercapai dan bot tidak mengirimkan pergerakannya untuk turn tersebut, maka tidak ada perintah yang dikirim ke server. Akibatnya, bot akan melewati turn tersebut. Jika bot melewati turn, ia tidak akan bisa menyesuaikan gerakannya atau menembakkan senjatanya karena server tidak menerima perintah tepat waktu sebelum turn berikutnya dimulai.

3. Energi

Semua bot memulai permainan dengan jumlah energi awal sebanyak 100 poin energi.

- Bot akan kehilangan energi jika ditembak atau ditabrak oleh bot musuh.
- Bot juga akan kehilangan energi jika menembakkan meriamnya.
- Bot akan mendapatkan energi jika peluru dari meriamnya mengenai musuh. Energi yang didapat akan lebih banyak 3 kali lipat dari energi yang digunakan untuk menembakkan peluru.

- Bot dengan energi nol akan dinonaktifkan dan tidak bisa bergerak. Jika bot terkena serangan dalam keadaan ini, bot akan hancur.

4. Peluru

Semakin banyak energi (daya tembak) yang digunakan untuk menembakkan peluru, semakin berat peluru tersebut dan semakin lambat gerakannya. Namun, peluru yang lebih berat juga menghasilkan lebih banyak kerusakan dan memungkinkan bot mendapatkan lebih banyak energi saat mengenai bot musuh.

Seperti disebutkan sebelumnya, peluru yang lebih berat akan bergerak lebih lambat. Ini berarti akan membutuhkan waktu lebih lama untuk mencapai target, meningkatkan risiko peluru tidak mengenai sasaran. Sebaliknya, peluru yang lebih ringan bergerak lebih cepat, sehingga lebih mudah mengenai target, tetapi peluru ringan tidak memberikan banyak poin energi saat mengenai bot musuh.

5. Panas Meriam (Gun Heat)

Saat menembakkan peluru, meriam akan menjadi panas. Peluru yang lebih berat menghasilkan lebih banyak panas dibandingkan peluru yang lebih ringan. Ketika meriam terlalu panas, bot tidak dapat menembak hingga suhu meriam turun ke nol. Selain itu, meriam juga sudah dalam keadaan panas di awal round dan perlu waktu untuk mendingin sebelum bisa digunakan untuk pertama kalinya.

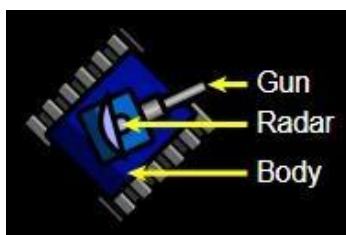
6. Tabrakan

Perlu diperhatikan bahwa bot akan menerima kerusakan jika menabrak dinding (batas arena), yang disebut wall damage. Hal yang sama juga terjadi jika bot bertabrakan dengan bot lain.

Jika bot menabrak bot musuh dengan bergerak maju, ini disebut ramming (menabrak dengan sengaja), yang akan memberikan sedikit skor tambahan bagi bot yang menyerang.

7. Bagian Tubuh Tank

Tubuh tank terdiri dari 3 bagian:



Gambar 2. Bagian Tubuh Tank

Body adalah bagian utama dari tank yang digunakan untuk menggerakkan tank.

Gun digunakan untuk menembakkan peluru dan dapat berputar bersama *body* atau independen dari *body*.

Radar digunakan untuk memindai posisi musuh dan dapat berputar bersama *body* atau independen dari *body*.

8. Pergerakan

Bot dapat bergerak maju dan mundur hingga kecepatan maksimum. Dibutuhkan beberapa giliran untuk mencapai kecepatan maksimum. Bot dapat mengalami percepatan maksimum sebesar 1 unit per giliran dan pengereman dengan perlambatan maksimum 2 unit per giliran. Percepatan dan perlambatan maksimum tidak bergantung pada kecepatan bot saat itu.

9. Berbelok

Seperti yang disebutkan sebelumnya, bagian tubuh, turret (meriam), dan radar dapat berputar secara independen satu sama lain. Jika turret atau radar tidak diputar, maka keduanya akan mengarah ke arah yang sama dengan tubuh bot.

Setiap bagian tubuh memiliki kecepatan putar yang berbeda. Radar adalah bagian tercepat dan dapat berputar hingga 45 derajat per giliran, yang berarti dapat berputar 360 derajat dalam 8 giliran. Turret dan meriam dapat berputar hingga 20 derajat per giliran.

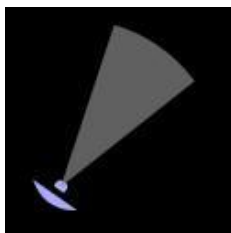
Bagian paling lambat adalah tubuh tank, yang dalam kondisi terbaik dapat berputar hingga 10 derajat per giliran. Namun, ini bergantung pada kecepatan bot saat ini. Semakin cepat bot bergerak, semakin lambat kemampuannya untuk berbelok.

Perlu diperhatikan bahwa tidak ada energi yang dikonsumsi saat bot bergerak atau berbelok.

10. Pemindaian

Aspek penting dalam Robocode adalah memindai bot musuh menggunakan radar. Radar dapat mendeteksi bot dalam jangkauan hingga 1200 piksel. Musuh yang berada lebih dari 1200 piksel dari bot tidak dapat terdeteksi atau dipindai oleh radar.

Penting untuk diperhatikan bahwa sebuah bot hanya dapat memindai bot musuh yang berada dalam jangkauan sudut pemindaian (scan arc)-nya. Sudut pemindaian ini merupakan "sapuan radar" dari arah radar sebelumnya ke arah radar saat ini dalam satu giliran.



Gambar 3. Sapuan Radar

Jika radar tidak bergerak dalam suatu giliran, artinya radar tetap mengarah ke arah yang sama seperti pada giliran sebelumnya, maka sudut pemindaian akan menjadi nol derajat, dan bot tidak akan dapat mendeteksi musuh.



Gambar 4. Pemindaian Radar Nol Derajat

Oleh karena itu, sangat disarankan untuk selalu mengubah arah radar agar tetap dapat memindai musuh.

11. Skor

Pada akhir pertempuran, setiap bot akan diranking berdasarkan total skor yang diperoleh masing-masing bot selama keseluruhan pertempuran. Tentunya, tujuan utama pada tugas besar ini adalah membuat bot yang memberikan skor setinggi mungkin. Berikut adalah rincian komponen skor pada pertempuran:

- **Bullet Damage:** Bot mendapatkan **poin sebesar *damage*** yang dibuat kepada bot musuh menggunakan peluru.
- **Bullet Damage Bonus:** Apabila peluru berhasil membunuh bot musuh, bot mendapatkan **poin sebesar 20% dari *damage*** yang dibuat kepada musuh yang terbunuh.
- **Survival Score:** Setiap ada bot yang mati, bot lainnya yang masih bertahan pada ronde tersebut mendapatkan **50 poin**.
- **Last Survival Bonus:** Bot terakhir yang bertahan pada suatu ronde akan mendapatkan **10 poin** dikali dengan banyaknya musuh.
- **Ram Damage:** Bot mendapatkan **poin sebesar 2 kalinya *damage*** yang dibuat kepada bot musuh dengan cara menabrak.
- **Ram Damage Bonus:** Apabila musuh terbunuh dengan cara ditabrak, bot mendapatkan **poin sebesar 30% dari *damage*** yang dibuat kepada musuh yang terbunuh.

Skor akhir bot adalah akumulasi dari 6 komponen diatas. Perlu diperhatikan bahwa game akan menampilkan berapa kali suatu bot meraih peringkat 1, 2, atau 3 pada setiap ronde. Namun, hal ini tidak dihitung sebagai komponen skor maupun untuk perangkingan akhir. Bot yang dianggap menang pertempuran adalah bot dengan akumulasi skor tertinggi.

BAB 2

LANDASAN TEORI

1. Algoritma Greedy

Dalam ilmu komputer Algoritma *greedy* merupakan salah satu algoritma yang paling sederhana dan paling banyak dipakai. Algoritma *greedy* merupakan sebuah algoritma yang memecahkan permasalahan dengan spontan dengan berharap solusi dari pemecahan masalah tersebut adalah solusi yang optimal.

Cara algoritma ini memecahkan masalah adalah dengan cara memecahkan permasalahan dengan melalui langkah per langkah, sehingga dengan masing-masing langkah mengambil pilihan terbaik yang ada dari solusi tanpa memikirkan konsekuensi kedepannya (*“take what you can get now”*), dengan harapan bahwa pilihan optimum lokal pada setiap langkah akan berakhir dengan optimum global. Namun, perlu diingat bahwa algoritma *greedy* tidak selalu memberikan sebuah solusi yang optimal, algoritma ini hanyalah memilih algoritma terbaik yang pertama kali muncul.

Solusi-solusi yang diberikan haruslah memenuhi persyaratan seperti berikut:

- *feasible*: Solusi harus memenuhi batasan batasan (*constraints*) yang ada dari permasalahan tersebut.
- *locally optimal*: Solusi haruslah pilihan yang paling optimal diantara semua pilihan yang memungkinkan dari semua solusi yang ada pada langkah tersebut.
- *irrevocable*: Ketika solusi telah ditemukan dan dibuat maka solusi tersebut tidak dapat diubah pada langkah-langkah selanjutnya pada algoritma tersebut.

Kegunaan algoritma *greedy* utamanya adalah untuk pemecahan persoalan optimasi atau persoalan mencari solusi optimal dari sebuah permasalahan yang membutuhkan solusi maksimum ataupun minimum dengan menggunakan sebuah cara yang efisien dan efektif.

Proses seleksi solusi dari persoalan biasanya terdiri dari langkah-langkah berikut:

- (1) Dimulai dengan solusi awal dari masalah (Solusi kosong).
- (2) Solusi saat ini secara bertahap diperluas melalui serangkaian pilihan optimal lokal.
- (3) Pada setiap langkah pilihan tersebut haruslah memenuhi sifat seleksi dari algoritma *greedy* yaitu, solusi saat ini haruslah solusi yang saat ini optimal.
- (4) Ulangi langkah (2) dan (3) hingga kondisi pemberhentian masalah terpenuhi.

Perlu diingat kembali ketika sebuah solusi optimal telah ditemukan maka algoritma tidak akan melakukan *backtrack*.

Elemen-elemen algoritma greedy:

1. Himpunan kandidat, C : berisi kandidat yang akan dipilih pada setiap Langkah (misal: simpul/sisi di dalam graf, job, task, koin, benda, karakter, dsb)
2. Himpunan solusi, S : berisi kandidat yang sudah dipilih
3. Fungsi solusi: menentukan apakah himpunan kandidat yang dipilih sudah memberikan solusi
4. Fungsi seleksi (selection function): memilih kandidat berdasarkan strategi greedy tertentu. Strategi greedy ini bersifat heuristik.
5. Fungsi kelayakan (feasible): memeriksa apakah kandidat yang dipilih dapat dimasukkan ke dalam himpunan solusi (layak atau tidak)
6. Fungsi obyektif : memaksimumkan atau meminimumkan

Dapat dikatakan bahwa algoritma *greedy* melibatkan pencarian sebuah himpunan bagian, S , dari himpunan kandidat, C ; yang dalam hal ini, S harus memenuhi beberapa kriteria yang ditentukan, yaitu S menyatakan suatu solusi dan S di optimisasi oleh fungsi objektif.

2. Cara Kerja Program Secara Umum

Bot pada Robocode bekerja dengan konsep *event-driven-programming*, yang artinya bot akan merespons dan melakukan aksi sesuai dengan apa yang telah terjadi pada sebelumnya di arena. Di dalam program terdapat beberapa fungsi yang menerapkan konsep *event-driven-programming* yaitu:

1. OnScannedBot(ScannedBotEvent e)
kondisi yang dipanggil ketika sebuah bot mendeteksi musuh, lalu akan menjalankan kode logika yang diterapkan pada fungsi ini.
2. OnHitByBullet(HitByBulletEvent e): Sebuah fungsi yang dipanggil ketika bot terkena tembakan oleh musuh maka, bot akan menjalankan kode yang ada di dalam fungsi ini.
3. OnHitWall(HitWallEvent e): Sebuah fungsi yang akan dipanggil ketika bot telah menabrak dinding dari arena dan akan langsung menjalankan kode program yang ada di dalam fungsi ini

Implementasi Algoritma *Greedy* pada bot:

Implementasi algoritma *greedy* pada program bot robocode berfokus pada pengambilan keputusan lokal optimal pada setiap putaran (tick), keputusan diambil berdasarkan solusi instan untuk memaksimalkan *survival rate*.

contoh pada bot adalah:

- Greedy dalam Manajemen energi
- Greedy dalam deteksi musuh
- Greedy dalam pemilihan peluru

BAB 3

APLIKASI STRATEGI GREEDY

3.1 Alternatif Greedy

3.1.1. Algoritma *Greedy* pada bot ambatron

Pendekatan Greedy yang dipakai oleh bot ini adalah ***Greedy by Survival with dodge***, yang mengutamakan opsi yang memaksimalkan solusi untuk memaksimalkan poin bertahan hidup pada setiap turn dengan memilih area aman dan menghindar.

1) Mapping Elemen *Greedy*

a) Himpunan Kandidat: Semua pilihan pergerakan yang telah dievaluasi bot, yang berisi 36 titik tujuan pada arena di sekeliling bot dengan radius 100 unit.

b) Himpunan Solusi: Satu titik atau area dengan nilai risiko minimum pada arena yang akan dijalankan oleh bot.

c) Fungsi Solusi:

Bot melakukan pengecekan dengan *scanning* arena baru setelah dilakukan *scanning*, maka bot akan bergerak ke titik yang telah dievaluasi dan diyakini aman. Aksi ini dilakukan secara tetap dan membuat bot melakukan *scanning* terus menerus dan jika bot mengalami serangan maka ia juga akan melakukan *dodge* dengan melakukan manuver menghindar reaktif secara instan.

d) Fungsi Seleksi: Bot memilih salah satu opsi terbaik dari 36 titik yang telah dievaluasi dan jika terdapat gangguan eksternal maka bot akan melakukan pergerakan instan.

e) Fungsi Kelayakan: Memeriksa titik pada arena yang paling aman dan jika arah menghindar dari peluru akan mengenai tembok maka fungsi akan menolaknya dan bergerak ke arah sebaliknya

f) Fungsi Objektif: Titik pada arena dengan nilai risiko minimal.

2) Analisis Efisiensi Solusi: Pada algoritma *greedy by survival with dodge*, bot memulai dengan *scanning* lalu mengevaluasi titik yang paling aman dari bot dengan jarak 100 unit, dan bot menggunakan perulangan untuk selalu mengecek data bot musuh yang aktif, ketika bot melakukan penghindaran bot langsung mengabaikan evaluasi titik aman dan langsung bergerak menghindar.

3) Analisis Efektivitas Solusi

Strategi ini efektif apabila:

- a) Bot lawan memiliki energi di ambang kritis
- b) Melawan Bot dalam kondisi satu vs satu
- c) Bot ditembaki peluru dari satu arah

Strategi ini tidak efektif apabila:

- a) Bot terjebak pada situasi ditembaki dari berbagai arah
- b) Bot lawan memiliki mekanisme prediksi pergerakan tingkat tinggi
- c) Terpojok pada sudut
- d) Bot lawan bergerak lincah

3.1.2. Algoritma Greedy pada bot blakutak

Pendekatan Greedy yang dipakai oleh bot ini adalah ***Greedy by movement***, yang mengutamakan pergerakan zigzag pada kecepatan maksimum untuk meminimumkan resiko dan penggunaan energi.

1) Mapping Elemen *Greedy*

- a) Himpunan Kandidat: Semua pilihan kecepatan dan arah putar pergerakan.
- b) Himpunan Solusi: Bot bergerak dengan kecepatan maksimum secara zigzag dan melakukan penghindaran dengan berbelok tegak lurus dari arah peluru saat tertembak.
- c) Fungsi Solusi: Bot bergerak dengan kecepatan maksimum bersamaan ketika permainan dimulai, lalu bot melakukan pengecekan melalui radar kemana bot harus bergerak jika menabrak dia segera mundur dan menghindar.
- d) Fungsi Seleksi: Memilih arah belok zig zag dan manuver penghindaran untuk membuat bot yang sulit diprediksi.
- e) Fungsi Kelayakan: Melakukan pengecekan jika pergerakan tidak menyebabkan bot keluar arena, dan memastikan penghindaran dapat dilakukan.
- f) Fungsi Objektif: Meminimalkan peluang terkena tembakan dan meminimalkan konsumsi energi.

2) Analisis Efisiensi Solusi: Dengan pendekatan algoritma *greedy by movement* seperti ini mula mula bot langsung bergerak dengan kecepatan maksimum, diikuti oleh pergerakan zig-zag, dalam pola perilaku seperti ini, bot terus menerus menghindari peluru dari lawan.

3) Analisis Efektivitas Solusi

Strategi ini efektif apabila:

- a) Bot lawan tidak mempunyai adaptasi pola.
- b) Bot lawan menggunakan mekanisme targeting sederhana.

Strategi ini tidak efektif apabila:

- e) Bot lawan memiliki mekanisme targeting yang kompleks
- f) Bot terjebak pada sudut arena
- g) Bot lawan merupakan bot bertipe agresif

3.1.3. Algoritma Greedy pada bot Gaspolbot

Pendekatan Greedy yang dipakai oleh bot ini adalah ***Greedy by Shoot on Sight with adaptive firepower***, yang mengutamakan

1) Mapping Elemen *Greedy*

- a) Himpunan Kandidat: Semua pilihan untuk daya tembak, dan pergerakan.
- b) Himpunan Solusi: Mengunci posisi senjata langsung ke arah musuh yang terdeteksi dan menembakkan peluru dengan daya tembakan sesuai dengan jarak antara musuh.
- c) Fungsi Solusi: Ketika bot berjalan, maka bot akan melakukan *scanning* menggunakan radar pada bot lain dan bot yang masuk ke dalam radar maka bot akan langsung menembakkan peluru sesuai dengan jarak tak peduli akan dengan harapan bot lain akan langsung hancur.
- d) Fungsi Seleksi: Memilih lawan dengan radar, siapapun yang masuk kedalam radar maka bot akan langsung bereaksi dengan daya pemilihan daya tembak 3, 2 atau 1.
- e) Fungsi Kelayakan: Mengecek berapa daya tembak yang sesuai dengan jarak.
- f) Fungsi Objektif: Meminimumkan efisiensi energi dan memaksimalkan kerusakan pada musuh berjarak dekat.

2) Analisis Efisiensi Solusi: Dengan pendekatan *greedy by shoot on sight with adaptive firepower*, dimulai dengan bot melakukan *scanning* lalu menembak satu bot yang terdeteksi pertama kali pada radar bot, hal ini dilakukan secara terus menerus pada setiap *tick*. algoritma ini berharap terdapat bot dalam jarak dekat sehingga bot dapat menembakkan dengan daya tembak 3.

3) Analisis Efektivitas Solusi

Strategi ini efektif apabila:

- a) Keadaan sudah 1 vs 1
- b) Musuh berada pada jarak dekat
- c) Musuh hanya diam

Strategi ini tidak efektif apabila:

- h) Terlalu banyak bot pada arena
- i) Musuh bergerak dengan lincah
- j) Musuh berada di luar jangkauan

3.1.4. Algoritma Greedy pada bot BotGajelas

Pada bot ini, pendekatan Greedy yang digunakan adalah ***Greedy by Distance***, yaitu strategi algoritma greedy yang fokus terhadap pemilihan target pada musuh yang berada di jarak yang paling dekat dengan bot. Tujuannya adalah untuk meminimalisir jarak tembak, sehingga dapat memaksimalkan kemungkinan peluru mengenai target dan mengurangi waktu musuh untuk dapat bereaksi.

1) Mapping Elemen *Greedy*

- a) Himpunan Kandidat: Himpunan kandidat pada bot ini adalah seluruh musuh yang pernah terdeteksi oleh radar dan keberadaannya masih tersimpan di memori bot. Setiap radar menyapu arena dan meng-*scan* musuh, data terakhir posisi mereka ditambahkan ke dalam daftar kandidat target yang dapat diserang.
- b) Himpunan Solusi: Bot memilih satu musuh terdekat untuk dijadikan target, lalu bot akan menjadikan target tersebut sebagai fokus utama, di mana semua pergerakan meriam, radar, dan laju bot akan ditujukan hanya untuk merespons keberadaan target spesifik tersebut.
- c) Fungsi Solusi: Mengevaluasi apakah proses pemilihan telah menghasilkan solusi yang siap dieksekusi. Jika fungsi seleksi dan fungsi kelayakan berhasil menemukan dan mengunci satu target yang valid, fungsi solusi akan mengonfirmasi keadaan tersebut dengan mengubah mode bot (misalnya, keluar dari mode rotasi radar acak/pencarian buta). Dengan ditemukannya solusi, bot secara optimal dapat mulai menerapkan strategi "zona aman" yaitu bermanuver maju, mundur, atau bergerak menyamping untuk menjaga jarak tembak ideal, serta melontarkan tembakan adaptif.
- d) Fungsi Seleksi: Membandingkan seluruh musuh yang terdapat di himpunan kandidat berdasarkan kriteria tertentu. Pada bot ini, kriteria utamanya adalah jarak absolut antara posisi bot dan posisi musuh. Fungsi seleksi akan mengabaikan musuh yang jauh dan secara spesifik memilih satu musuh terdekat.

- e) Fungsi Kelayakan: Memastikan musuh tersebut masih hidup (jika musuh hancur, maka dicoret dari daftar). Lalu memastikan data pelacakan musuh masih baru. Jika bot tidak berhasil memindai ulang target tersebut dalam rentang waktu toleransi tertentu (melewati batas ambang batas kehilangan jejak), target dianggap tidak lagi layak, dan bot akan melepaskan fokusnya untuk mencari target baru yang lebih pasti
 - f) Fungsi Objektif: Meminimumkan jarak antara dirinya dan ancaman yang akan dihadapi. Bot meminimumkan objektif jarak pada tiap giliran, bot memaksimalkan peluang akurasi tembakan proyektil. Target yang paling dekat adalah target yang paling mudah dihitung prediksinya dan paling sedikit memberikan waktu bagi musuh untuk menghindar.
- 2) Analisis Efisiensi Solusi: Dengan pendekatan *greedy by distance*, dimulai dengan bot melakukan *scanning* untuk mengumpulkan data posisi musuh lalu langsung mengunci satu target yang berada pada jarak terdekat, hal ini dievaluasi secara terus-menerus pada setiap *tick*. Algoritma ini berharap target yang dikunci selalu berada atau bergerak di sekitar rentang jarak zona aman (150 hingga 280), sehingga bot dapat secara efisien melakukan manuver menyamping (*strafing*) untuk menghindari peluru lawan sekaligus memberikan tembakan adaptif yang akurat tanpa perlu membuang banyak waktu dan pergerakan untuk terus mengejar atau mundur.
- 3) Analisis Efektivitas Solusi
- Strategi ini efektif apabila:
- a) Pertarungan 1vs1
 - b) Melawan bot tipe penabrak (rammer)
 - c) Musuh memiliki pergerakan lambat
 - d) Arena yang sempit
 - e) Menghadapi musuh dengan *firepower* lemah
- Strategi ini tidak efektif apabila:
- a) Arena ramai
 - b) Menghadapi musuh jarak jauh
 - c) Terjepit di sudut atau dinding
 - d) Musuh memiliki *energy* rendah di jarak jauh
 - e) Menghadapi bot dengan *advanced targeting*

3.2 Strategi Greedy yang Dipilih

Strategi *greedy* yang akan dipilih dan akan digunakan oleh kelompok kami adalah algoritma *greedy by survival with dodge*. Strategi ini berfokus memaksimalkan nilai survivalitas sampai akhir ronde.

Alasan kami memilih untuk menggunakan algoritma tersebut dikarenakan fungsi untuk melakukan pergerakan ke areal aman dengan nilai risiko minimal yang telah dipilih dari evaluasi 36 titik, dan juga sistem reaktif *dodge* yang memungkinkan bot untuk bergerak secara reaktif atas suatu kejadian yang terjadi jika terdapat penyerangan terhadap dirinya, bot akan langsung mem-*bypass* fungsi evaluasi area dengan nilai risiko minimal dan langsung memaksa bergerak 90°.

Pada bot ini juga terdapat fungsi yang akan menerapkan agresivitas terhadap bot yang memiliki sisa energi pada ambang kritis, bot akan langsung memburu bot tersebut untuk mendapatkan energi.

BAB 4

IMPLEMENTASI DAN PENGUJIAN

4.1 Implementasi Greedy pada Pseudocode

Greedy by Survival (ambatron)

Inisialisasi memori riwayat musuh dan independensi radar/gun

Selama bot berjalan:

// --- Sistem Pergerakan ---

Jika sedang dalam mode menghindar (evade):

Bergerak darurat sesuai arah menghindar

Jika melawan 1 musuh (1v1):

Arah = 30% Cari posisi paling aman (Greedy) + 70% Mengorbit target (Circling)

Jika melawan banyak musuh:

Arah = 100% Mengorbit target (Circling)

Kalkulasi efisiensi gerak: mundur jika bot harus berputar lebih dari 90°

Pindah ke Arah yang sudah ditentukan

// --- Sistem Menembak ---

Jika target memiliki riwayat dan data masih baru:

Arahkan senjata (gun) ke posisi terakhir target

Jika senjata sudah sejajar dengan target:

power <- tembakan besar jika sangat dekat, mengecil jika jauh

Tembak dengan power

// --- Sistem Pemindaian (Scan) ---

Jika target hilang terlalu lama ATAU sedang tidak ada target:

 Aktifkan Mode Radar (berputar 360° penuh)

Jika target ada:

 Aktifkan Mode Fokus (sapu radar bolak-balik dalam sudut sempit ke arah target)

Jalankan perintah (Go)

Ketika mendeteksi musuh (OnScannedBot):

 Simpan/perbarui data musuh ke memori riwayat

 Jika musuh ini lebih dekat dari target saat ini:

 Jadikan musuh ini sebagai target utama

 Jika sedang memindai penuh (Mode Radar) dan target terdeteksi:

 Langsung beralih ke Mode Fokus

Ketika tertembak peluru (OnHitByBullet):

 Balik arah saat mengorbit musuh

 Aktifkan mode menghindar (bergerak tegak lurus dari arah peluru)

Ketika menabrak dinding (OnHitWall):

 Balik arah saat mengorbit musuh

 Aktifkan mode menghindar (bergerak darurat menuju tengah arena)

Greedy by movement (blakutak)

Inisialisasi warna bot (Biru, Hitam, Biru)

turnCounter $\leftarrow$ 0

GunTurnRate $\leftarrow$ 20

Selama bot berjalan:

 TargetSpeed $\leftarrow$ 8 (kecepatan maksimum)

 Jika (turnCounter modulo 40) < 20:

 TurnRate $\leftarrow$ 10 (belok tajam ke kiri)

 Sebaliknya:

 TurnRate $\leftarrow$ -10 (belok tajam ke kanan)

 turnCounter $\leftarrow$ turnCounter + 1

 Jalankan perintah (Go)

Ketika mendeteksi musuh (OnScannedBot):

Tembak dengan kekuatan 1 (hemat energi)

Ketika terkena peluru (OnHitByBullet):

bearing $\leftarrow$ arah datangnya peluru relatif terhadap bot

Jika bearing < 0 :

TurnRate $\leftarrow 10$ (menghindar ke kiri)

Sebaliknya:

TurnRate $\leftarrow -10$ (menghindar ke kanan)

TargetSpeed $\leftarrow 8$

Ketika menabrak dinding (OnHitWall):

TargetSpeed $\leftarrow -8$ (mundur dengan kecepatan penuh)

TurnRate $\leftarrow 10$ (belok tajam untuk keluar)

Greedy by Shoot on Sight with Adaptive Firepower (gaspolbot)

Inisialisasi warna bot (Merah, Oranye, Merah)

movingForward $\leftarrow$ true

Selama bot berjalan:

Bergerak maju sejauh 40000

movingForward $\leftarrow$ true

Putar bot ke kanan 90 derajat, tunggu hingga selesai

Putar bot ke kiri 180 derajat, tunggu hingga selesai

Putar bot ke kanan 180 derajat, tunggu hingga selesai

Fungsi BalikArah():

Jika movingForward adalah true:

Bergerak mundur sejauh 40000

movingForward $\leftarrow$ false

Sebaliknya:

Bergerak maju sejauh 40000

movingForward $\leftarrow$ true

Ketika mendeteksi musuh (OnScannedBot):

Arahkan pistol tepat ke posisi musuh

jarak $\leftarrow$ hitung jarak dari bot ke musuh

Jika jarak < 150 :

firePower $\leftarrow 3$

Sebaliknya jika jarak < 300 :

firePower $\leftarrow 2$

Sebaliknya:

firePower $\leftarrow 1$

Tembak peluru dengan kekuatan firePower

Ketika menabrak dinding (OnHitWall):

Jalankan BalikArah()

Ketika menabrak bot lain (OnHitBot):

Jika bot ditabrak oleh musuh:

Jalankan BalikArah()

Greedy by Distance (botgajelas)

Inisialisasi awal bot (warna kuning, pisahkan gerak radar, meriam, dan badan bot)

Selama bot berjalan:

Hitung akumulasi putaran radar

Jika sedang mode menyapu (sweeping):

Jika ada data musuh di memori:

Arahkan radar ke posisi musuh terdekat di memori

Jika tidak:

Putar radar perlahan untuk mencari (blind sweep)

Jika sedang reposisi:

Arahkan dan gerakkan bot menuju tengah arena

Jika waktu reposisi selesai -> hentikan reposisi

Jika radar sudah berputar satu lingkaran penuh (360 derajat):

Balikkan arah putaran radar

Mulai reposisi ke tengah arena

Jika tidak sedang menyapu (sedang fokus ke target):

Jika target tidak terdeteksi selama beberapa waktu:

Kembali ke mode menyapu (sweeping)

Balikkan arah menghindar (strafe) setiap 25 putaran waktu (ticks)

Jalankan perintah pergerakan (Go)

Saat mendeteksi musuh (OnScannedBot):

Simpan/update posisi dan jarak musuh ke dalam memori

target <- musuh terdekat dari semua memori yang ada

Hentikan mode menyapu (sweeping)

Jika musuh yang dideteksi = target:

Kunci radar ke arah target

Arahkan meriam ke target

// Pertahankan jarak aman

Jika jarak < batas minimal:

 Mundur menjauhi target

Jika jarak > batas maksimal:

 Maju mendekati target

Jika jarak di area aman:

 Bergerak menyamping (strafing) mengelilingi target

Jika meriam sudah lurus dengan target:

 Tembak target (daya ledak menyesuaikan jarak: makin dekat, makin besar)

Saat ada bot yang mati (OnBotDeath):

 Hapus bot tersebut dari memori

 Jika bot yang mati adalah target:

 Reset memori target dan kembali ke mode menyapu (sweeping)

Saat menabrak dinding (OnHitWall) atau menabrak bot (OnHitBot):

 Balikkan arah menghindar (strafe)

 Mundur ke belakang dan putar badan bot untuk menghindar

4.2 Penjelasan Struktur Algoritma *Greedy by Survival with dodge*

1) Penjelasan struktur data yang ada pada algoritma ini:

a) Point

 berguna untuk menyimpan koordinat, untuk merekam posisi musuh

b) Bot History Entry

 Wadah untuk mencatat sisa energi musuh, posisi dan arah gerak bot

c) Dictionary<string>

 untuk menyimpan nama musuh dan menyimpannya sebagai daftar.

2) Penjelasan fungsi yang ada pada algoritma ini:

a) CalculateGreedyDirection()

 Fungsi utama dari algoritma *greedy* ini fungsi ini berfungsi memetakan 36 titik kandidat di sekeliling bot.

b) EvaluateRisk()

 Fungsi yang menghitung total risiko untuk suatu area berdasarkan *wall risk*, *center risk*, dan *enemy risk*.

c) GetTarget()

 Fungsi untuk mencari data dari *enemyHistory* untuk mencari nama dan mengembalikan nama musuh yang dapat diserang.

- d) `NormalizeAbsoluteAngle()` & `NormalizeRelativeAngle()`
Fungsi matematika untuk menormalisasi sudut dalam derajat untuk mencegah error pada perputaran radar dan bot.

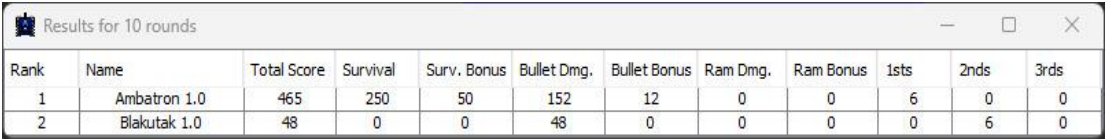
3) Penjelasan prosedur yang ada pada algoritma ini:

- a) `Run()`
Prosedur utama yang dijalankan sejak awal ronde, bertindak sebagai wadah prosedur lain.
- b) `HandleMovement()`
Berguna mengatur pergerakan fisik robot, mengatur arah gerak bot. Prosedur ini juga yang menentukan kapan melakukan dodge.
- c) `HandleGun()`
Berguna mengatur arah orientasi meriam berdasarkan kalkulasi dan prediksi tembakan serta menentukan daya tembak secara dinamis.
- d) `HandleRadar()`
Berguna mengatur arah rotasi radar berdasarkan `ScanMode`.
- e) `OnScannedBot(ScannedBotEvent e)`
Prosedur ketika radar mendeteksi musuh, tugas utamanya adalah memperbarui intelijen dan membuat objek bagi `BotHistoryEntry`.
- f) `OnHitByBullet(HitBYBulletEvent e)`
Prosedur yang akan dijalankan ketika bot terkena peluru bot lawan dari prosedur inilah bot akan menetapkan `evadeTimer` yang akan mengaktifkan sistem Dodge.
- g) `OnHitWall(HitWallEvent e)`
Prosedur interupsi jika bot mengenai dinding.

4.3 Pengujian Perbandingan antar Bot dalam Pertandingan

Dalam pengujian ini kami melakukan pertandingan 1 vs 1 dengan bot utama dan bot alternatif.

4.3.1 Ambatron vs Blakutak



Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Ambatron 1.0	465	250	50	152	12	0	0	6	0	0
2	Blakutak 1.0	48	0	0	48	0	0	0	0	6	0

Bot utama Ambatron melawan bot blakutak, bot ambatron menang jauh dengan total poin 465 dari 523 poin total, hal ini menandakan memang strategi algoritma dari bot ambatron berhasil diterapkan dikarenakan kondisi 1 vs 1, dan hal ini juga yang menjadi alasan bot Blakutak kalah, dikarenakan kelemahan pada strategi algoritma

greedy by movement adalah jika bertemu lawan dengan sistem locking satu musuh dan kondisi 1 vs 1.

4.3.2 Ambatron vs GaspolBot

Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Ambatron 1.0	343	150	30	144	19	0	0	3	1	0
2	Gaspolbot 1.0	103	50	10	42	0	1	0	1	3	0

Bot Ambatron dan GaspolBot sama-sama unggul dalam *Survivalbility*, dilihat dari skor keduanya mendapatkan *Survival Bonus*, walaupun pendekatan algoritma pada GaspolBot adalah *Greedy by Shoot on Sight with adaptive firepower*, dimana walaupun keduanya unggul dalam *Survivalbility* akan tetapi tetap akan kalah saing dengan Ambatron yang menggunakan pendekatan algoritma *Greedy by Survival with Dodge*, yang memang benar-benar mengutamakan *Survivability* dengan cara bagaimana mekanisme *Dodge* bergerak secara acak agar posisi sulit ditebak.

4.3.3 Ambatron vs BotGajelas

Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Ambatron 1.0	1068	400	80	496	92	0	0	9	0	0
2	BotGajelas 1.0	150	0	0	150	0	0	0	0	9	0

Bot Ambatron menang dan mendominasi pertandingan dalam segala aspek dikarenakan strateginya dioptimalkan dalam kondisi 1 vs 1 melalui pergerakan *hybrid* yang susah untuk diprediksi dan respons agresif terhadap musuh sekerat. Sebaliknya, BotGajelas kalah karena algoritmanya yang terlalu statis, dimana penjagaan jaraknya yang kaku dan mudah untuk ditebak, membuat BotGajelas tidak dapat mengimbangi kelincihan dan kemampuan adaptasi Ambatron di arena.

BAB 5

KESIMPULAN

5.1 Kesimpulan

Berdasarkan hasil percobaan yang dilakukan dan pengujian yang telah dilakukan dengan membuat 4 bot, 1 bot sebagai bot utama dan 3 bot lain sebagai bot alternatif pada Robocode tank royale, dapat disimpulkan bahwa, strategi algoritma *greedy* memang dapat diaplikasikan dan merupakan pendekatan yang efektif dengan konsep “*take what you can get now*” yang berarti bot akan melakukan aksi optimal yang tersedia pada saat itu tanpa mengkhawatirkan penyebab yang akan terjadi.

Dengan algoritma *greedy* masing-masing bot dapat melakukan aksi optimal berdasarkan pilihan yang tersedia pada setiap *turn*-nya dan menghasilkan hasil yang optimum dimulai dari pergerakan, daya tembak dan jenis peluru.

Jika bot tidak dapat mengaplikasikan sebuah algoritma *greedy* yang ada merupakan memang kelemahan ketika menemui lawan yang memang kelemahan alami dari algoritma *greedy* yang dipakai. itulah kelemahan dari algoritma *greedy* yang mengutamakan solusi optimum pada saat itu tanpa mengkhawatirkan hasil yang akan terjadi pada masa depan.

Akan tetapi secara keseluruhan, masing masing bot yang telah diciptakan telah menerapkan strategi algoritma untuk masing masing bot, dimulai dari *greedy by survival with dodge*, *greedy by movement*, *greedy by distance* dan *greedy by shoot on sight with adaptive firepower*.

5.2 Saran

Pembuatan dan pengembangan bot pada Robocode dapat menggunakan pendekatan multi algoritma dan tidak hanya terpaku pada satu strategi algoritma, hal itu juga dapat memperbesar potensi kemenangan dari suatu bot dengan penggunaan algoritma sesuai dengan kondisi yang terjadi pada arena pertandingan atau bahkan memprediksi pergerakan lawan maupun aksi yang akan diambil berdasarkan multi algoritma yang dipakai.

LAMPIRAN DAN DAFTAR PUSTAKA

No	Poin	Ya	Tidak
1	Bot dapat dijalankan pada Engine yang sudah dimodifikasi asisten.	✓	
2	Membuat 4 solusi greedy dengan heuristic yang berbeda.	✓	
3	Membuat laporan sesuai dengan spesifikasi.	✓	
4	Membuat video bonus dan diunggah pada Youtube.		✓

Repository Github:

https://github.com/fvndee/Tubes_OMKEGAS

Daftar Pustaka:

Munir, R. Algoritma Greedy bagian 1,

[https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/04-Algoritma-Greedy-\(2026\)-Bag1.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/04-Algoritma-Greedy-(2026)-Bag1.pdf)

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein.
(2022). Introduction to Algorithms Fourth Edition. MIT Press

Anany Levitin. (2011). Introduction to The Design and Analysis of Algorithms.
Addison Wesley.

Yizhun Wang. (2023, November). Review on greedy algorithm. *Review on greedy algorithm*, (Greedy Algorithm), 7. 10.54254/2753-8818/14/2024104.